

BANGLADESH ECOMMERCE PLATFORM

Full-Scale Technical Specification & AI Coding Prompt Guide

Version	1.0
Stack	Next.js · NestJS · PostgreSQL · Redis
Payments	bKash · Nagad · SSL Commerz · COD
Delivery	Pathao · Steadfast · Redx · Paperfly
Timeline	12–18 months (3–5 dev team)

Table of Contents

- 1. Project Overview & Goals
- 2. System Architecture
- 3. Database Schema Design
- 4. Backend API Endpoints
- 5. Frontend Structure (Next.js)
- 6. Payment Integration (bKash, Nagad, SSL Commerz)
- 7. Delivery Partner Integration
- 8. Authentication & Security
- 9. Search & Product Discovery
- 10. Seller Dashboard Features
- 11. Admin Panel Features
- 12. Mobile App (React Native)
- 13. AI Coding Prompts (Copy-Paste Ready)
- 14. Infrastructure & DevOps
- 15. Environment Variables & Config
- 16. Testing Strategy
- 17. Launch Checklist

1. Project Overview & Goals

You are building a multi-vendor ecommerce marketplace for Bangladesh — similar to Amazon, Daraz, or Shajgoj — where third-party sellers list products and buyers purchase them. The platform must handle Bangladesh-specific requirements: COD (Cash on Delivery), bKash/Nagad mobile banking, local courier APIs, and Bangla language support.

Core Platform Type

- Multi-vendor marketplace (sellers + buyers + admin)
- Supports 100,000+ products, 10,000+ sellers, 1M+ buyers at scale
- Mobile-first (95% of BD internet traffic is mobile)
- Bilingual: Bangla (বাংলা) + English
- COD-first payment model with digital payment options

User Roles

Role	Access	Key Features
Buyer	Public + Auth	Browse, cart, checkout, track orders, review products
Seller	Auth + Dashboard	List products, manage inventory, view orders, analytics
Admin	Full access	Approve sellers, manage disputes, platform analytics, ban users
Delivery Staff	Limited	Update shipment status, confirm delivery

SECTION 2

2. System Architecture

Use a monorepo structure with microservice-ready architecture. Start as a monolith, split into services when traffic demands it. All services communicate via REST API (and WebSockets for real-time features).

Recommended Folder Structure

```
/ecommerce-bd
/apps
/web ← Next.js buyer frontend
/seller ← Next.js seller dashboard
/admin ← Next.js admin panel
/mobile ← React Native app
/api ← NestJS backend (main API)
/packages
/ui ← Shared components
/types ← Shared TypeScript types
/utils ← Shared utilities
/infra
/docker ← Docker configs
/nginx ← Reverse proxy
/k8s ← Kubernetes manifests (scale later)
docker-compose.yml
package.json ← Turborepo root
```

Tech Stack (Full)

Layer	Technology	Why
Buyer Frontend	Next.js 14 + App Router + Tailwind	SSR for SEO, fast loading on mobile
Seller Dashboard	Next.js 14 + React Query	SPA feel with server-side auth
Admin Panel	Next.js 14 + shadcn/ui	Pre-built components, rapid development
Mobile App	React Native + Expo	Single codebase for iOS + Android
Backend API	NestJS (Node.js)	TypeScript, decorators, scalable modules
Primary DB	PostgreSQL 15	Relational data, ACID transactions
Cache / Queue	Redis 7	Session cache, job queues, rate limiting
Search Engine	Meilisearch	Instant search, typo-tolerant, Bangla support
File Storage	AWS S3 / Wasabi	Product images, documents (Wasabi is 80% cheaper)

CDN	Cloudflare	Image delivery, DDoS protection, free SSL
Email	SendGrid / Resend	Transactional emails, OTP, notifications
SMS/OTP	SSL Wireless / Twilio	Phone verification, order SMS in Bangladesh
Push Notifications	Firebase FCM	Mobile push, web push notifications
Auth	JWT + Refresh Tokens	Stateless auth with secure refresh
Payments	SSL Commerz / AmarPay	Supports bKash, Nagad, VISA, Mastercard
Process Manager	PM2 / Docker	Zero-downtime deploys
CI/CD	GitHub Actions	Automated test + deploy pipeline
Monitoring	Sentry + Grafana	Error tracking + metrics dashboard
Hosting	AWS EC2 / DigitalOcean	VPS with auto-scaling

SECTION 3

3. Database Schema Design

Use PostgreSQL with TypeORM (NestJS) or Prisma. All tables use UUID primary keys. Use soft-delete (deleted_at timestamp) instead of hard delete for buyers, sellers, orders.

Table: users

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
name VARCHAR(100) NOT NULL
phone VARCHAR(20) UNIQUE NOT NULL
email VARCHAR(150) UNIQUE
password_hash TEXT
role ENUM('buyer', 'seller', 'admin') DEFAULT 'buyer'
avatar_url TEXT
is_verified BOOLEAN DEFAULT false
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ DEFAULT NOW()
deleted_at TIMESTAMPTZ
```

Table: sellers

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id UUID REFERENCES users(id)
shop_name VARCHAR(200) NOT NULL
shop_slug VARCHAR(200) UNIQUE NOT NULL
nid_number VARCHAR(30)
trade_license VARCHAR(50)
kyc_status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending'
commission_rate DECIMAL(5,2) DEFAULT 10.00
balance DECIMAL(12,2) DEFAULT 0.00
bank_account_name TEXT
bank_account_number TEXT
bank_name TEXT
bkash_number VARCHAR(20)
created_at TIMESTAMPTZ DEFAULT NOW()
```

Table: categories

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
name_en VARCHAR(100) NOT NULL
```

```

name_bn VARCHAR(100)
slug VARCHAR(100) UNIQUE NOT NULL
parent_id UUID REFERENCES categories(id)
image_url TEXT
sort_order INT DEFAULT 0
is_active BOOLEAN DEFAULT true

```

Table: products

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
seller_id UUID REFERENCES sellers(id)
category_id UUID REFERENCES categories(id)
name_en VARCHAR(300) NOT NULL
name_bn VARCHAR(300)
slug VARCHAR(300) UNIQUE NOT NULL
description_en TEXT
description_bn TEXT
base_price DECIMAL(12,2) NOT NULL
sale_price DECIMAL(12,2)
stock_quantity INT DEFAULT 0
sku VARCHAR(100)
weight_grams INT
status ENUM('draft','active','paused','banned') DEFAULT 'draft'
is_featured BOOLEAN DEFAULT false
views_count INT DEFAULT 0
sales_count INT DEFAULT 0
created_at TIMESTAMPTZ DEFAULT NOW()

```

Table: product_images

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
product_id UUID REFERENCES products(id) ON DELETE CASCADE
url TEXT NOT NULL
sort_order INT DEFAULT 0
is_primary BOOLEAN DEFAULT false

```

Table: product_variants

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
product_id UUID REFERENCES products(id) ON DELETE CASCADE
name VARCHAR(100) -- e.g. 'Red / XL'
sku VARCHAR(100)

```

```
price_modifier DECIMAL(10,2) DEFAULT 0
stock_quantity INT DEFAULT 0
attributes JSONB -- {color: 'red', size: 'XL'}
```

Table: orders

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
order_number VARCHAR(30) UNIQUE NOT NULL
buyer_id UUID REFERENCES users(id)
status ENUM('pending','confirmed','processing',
'shipped','delivered','cancelled','returned') DEFAULT 'pending'
payment_method ENUM('cod','bkash','nagad','card','rocket')
payment_status ENUM('unpaid','paid','refunded') DEFAULT 'unpaid'
subtotal DECIMAL(12,2) NOT NULL
delivery_fee DECIMAL(10,2) DEFAULT 0
discount_amount DECIMAL(10,2) DEFAULT 0
total DECIMAL(12,2) NOT NULL
delivery_address_id UUID REFERENCES addresses(id)
notes TEXT
created_at TIMESTAMPTZ DEFAULT NOW()
```

Table: order_items

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
order_id UUID REFERENCES orders(id) ON DELETE CASCADE
product_id UUID REFERENCES products(id)
variant_id UUID REFERENCES product_variants(id)
seller_id UUID REFERENCES sellers(id)
quantity INT NOT NULL
unit_price DECIMAL(12,2) NOT NULL
total_price DECIMAL(12,2) NOT NULL
commission_amount DECIMAL(10,2)
seller_earning DECIMAL(10,2)
```

Table: addresses

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id UUID REFERENCES users(id)
label VARCHAR(50) -- 'Home', 'Office'
recipient_name VARCHAR(100)
phone VARCHAR(20)
division VARCHAR(50)
```



```

district VARCHAR(50)
upazila VARCHAR(50)
street_address TEXT
postal_code VARCHAR(10)
is_default BOOLEAN DEFAULT false

```

Table: reviews

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
product_id UUID REFERENCES products(id)
buyer_id UUID REFERENCES users(id)
order_item_id UUID REFERENCES order_items(id)
rating INT CHECK (rating BETWEEN 1 AND 5)
title VARCHAR(200)
body TEXT
images TEXT[]
is_verified_purchase BOOLEAN DEFAULT true
created_at TIMESTAMPTZ DEFAULT NOW()

```

Table: coupons

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
code VARCHAR(50) UNIQUE NOT NULL
type ENUM('percentage', 'fixed') NOT NULL
value DECIMAL(10,2) NOT NULL
min_order_amount DECIMAL(10,2) DEFAULT 0
max_discount DECIMAL(10,2)
usage_limit INT
used_count INT DEFAULT 0
starts_at TIMESTAMPTZ
expires_at TIMESTAMPTZ
is_active BOOLEAN DEFAULT true

```

Table: payments

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid()
order_id UUID REFERENCES orders(id)
method ENUM('cod', 'bkash', 'nagad', 'card', 'rocket')
gateway_transaction_id VARCHAR(200)
amount DECIMAL(12,2) NOT NULL
currency VARCHAR(5) DEFAULT 'BDT'
status ENUM('pending', 'completed', 'failed', 'refunded')

```

```
gateway_response JSONB
created_at TIMESTAMPTZ DEFAULT NOW()
```

Table: shipments

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
order_id UUID REFERENCES orders(id)
courier ENUM('pathao','steadfast','redx','paperfly','sundarban')
tracking_id VARCHAR(200)
status VARCHAR(100)
pickup_date TIMESTAMPTZ
estimated_delivery TIMESTAMPTZ
delivered_at TIMESTAMPTZ
courier_response JSONB
```

Table: notifications

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id UUID REFERENCES users(id)
type VARCHAR(100) -- 'order_placed', 'shipped', etc.
title VARCHAR(200)
body TEXT
data JSONB
is_read BOOLEAN DEFAULT false
created_at TIMESTAMPTZ DEFAULT NOW()
```

SECTION 4

4. Backend API Endpoints

All endpoints under /api/v1. Use JWT Bearer token for authenticated routes. Return standard JSON: { success, data, message, pagination? }

AUTH

Method	Endpoint	Description
POST	/auth/register	Register with phone + OTP verify
POST	/auth/login	Login with phone + password
POST	/auth/otp/send	Send OTP to phone
POST	/auth/otp/verify	Verify OTP and return token
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Invalidate refresh token
GET	/auth/me	Get current user profile

PRODUCTS

Method	Endpoint	Description
GET	/products	List products (filter, sort, paginate)
GET	/products/:slug	Get single product with variants/reviews
GET	/products/search	Full-text search (Meilisearch)
GET	/products/category/:slug	Products by category
GET	/products/featured	Featured/promoted products
POST	/products	Create product (seller auth)
PATCH	/products/:id	Update product (seller auth)
DELETE	/products/:id	Soft-delete product
POST	/products/:id/images	Upload product images

ORDERS

Method	Endpoint	Description
GET	/orders	List buyer's orders

POST	/orders	Create order (checkout)
GET	/orders/:id	Get order details + tracking
PATCH	/orders/:id/cancel	Cancel order (before shipped)
POST	/orders/:id/return	Request return/refund
GET	/seller/orders	List seller's orders
PATCH	/seller/orders/:id/status	Update order status (seller)
GET	/admin/orders	All orders (admin)

PAYMENTS

Method	Endpoint	Description
POST	/payments/bkash/initiate	Start bKash payment
POST	/payments/bkash/callback	bKash payment webhook
POST	/payments/nagad/initiate	Start Nagad payment
POST	/payments/nagad/callback	Nagad webhook
POST	/payments/sslcommerz/initiate	SSL Commerz initiation
POST	/payments/sslcommerz/success	Payment success callback
POST	/payments/sslcommerz/fail	Payment fail callback
POST	/payments/sslcommerz/ipn	IPN webhook
GET	/payments/:orderId	Get payment status

DELIVERY

Method	Endpoint	Description
POST	/delivery/pathao/create	Create Pathao shipment
GET	/delivery/pathao/track/:id	Track Pathao shipment
POST	/delivery/steadfast/create	Create Steadfast shipment
GET	/delivery/steadfast/track/:id	Track Steadfast shipment
POST	/delivery/redx/create	Create Redx shipment
GET	/delivery/rate	Get delivery rate estimate

SELLERS

Method	Endpoint	Description
POST	/seller/register	Apply as seller
GET	/seller/dashboard	Seller dashboard stats
GET	/seller/products	List own products
GET	/seller/analytics	Sales analytics + charts
GET	/seller/earnings	Earnings + payout history
POST	/seller/payout/request	Request payout
GET	/seller/shop/:slug	Public seller shop page

ADMIN

Method	Endpoint	Description
GET	/admin/sellers/pending	Sellers awaiting KYC approval
PATCH	/admin/sellers/:id/approve	Approve seller KYC
PATCH	/admin/sellers/:id/reject	Reject seller KYC
GET	/admin/analytics	Platform-wide analytics
POST	/admin/categories	Create/edit category
POST	/admin/flash-sales	Create flash sale campaign
DELETE	/admin/products/:id	Remove product (admin)
POST	/admin/payouts/process	Process seller payouts

5. Frontend Structure (Next.js 14)

Buyer Web App Pages

/

Homepage — hero banner, featured categories, flash sale, recommended products

/search

Search results with filters (price, category, rating, brand, location)

/category/[slug]

Category page with subcategory sidebar, product grid

/product/[slug]

Product detail — images, variants, add to cart, reviews

/cart

Cart page — quantity edit, coupon code, delivery estimate

/checkout

Checkout — address, payment method (COD/bKash/card), order review

/checkout/success

Order confirmation with tracking number

/account/orders

Order history with status tracking

/account/orders/[id]

Single order detail + live tracking

/account/profile

Edit profile, addresses, change password

/account/wishlist

Saved/wishlisted products

/seller/[slug]

Public seller shop page

/auth/login

Login with phone OTP or password

/auth/register

Register buyer account

/track/[tracking_id]

Public order tracking page (no login needed)

/flash-sale

Flash sale campaign page with countdown timer

/[category]

Top-level category landing page

Key Frontend Components

- ProductCard — image, name, price, rating, add-to-cart button
- ProductGallery — swipeable image carousel with zoom
- VariantSelector — color/size chips, stock indicator
- CartDrawer — slide-in cart panel with quantity controls
- CheckoutStepper — Address → Payment → Review → Confirm
- OrderTracker — timeline showing order status with courier tracking
- FlashSaleTimer — countdown banner with live stock depletion
- SearchBar — instant search with autocomplete (Meilisearch)
- CategoryMegaMenu — hover dropdown with subcategories + images
- ReviewList — paginated reviews with star filter
- BanglaToggle — switch UI language between  and English
- CODConfirmModal — extra confirmation step for Cash on Delivery
- AddressForm — Bangladesh districts/upazilas dropdown (pre-populated)

6. Payment Integration

SSL Commerz (Primary Gateway)

SSL Commerz is the most widely used payment gateway in Bangladesh. It supports bKash, Nagad, Rocket, VISA, Mastercard, and other methods through a single integration. Apply at: sslcommerz.com — approval takes 2–5 business days.

SSL Commerz Integration Steps:

1. Register at developer.sslcommerz.com and get `store_id` + `store_passwd`
2. Install: `npm install sslcommerz-lts`
3. Set environment variables: `SSLCOMMERZ_STORE_ID`, `SSLCOMMERZ_STORE_PASSWD`, `SSLCOMMERZ_IS_LIVE`
4. On checkout: POST to SSL Commerz with order total, customer info, callback URLs
5. SSL Commerz returns a redirect URL — send buyer there to complete payment
6. SSL Commerz posts to your `/payments/sslcommerz/success` endpoint on completion
7. Verify payment with SSL Commerz validation API before marking order as paid
8. Also handle `/payments/sslcommerz/fail` and `/payments/sslcommerz/cancel` endpoints
9. Set up IPN (Instant Payment Notification) webhook for backend verification

bKash Direct API (Optional - Higher Volume)

1. Apply at developer.bka.sh for merchant API access (requires trade license)
2. Get: `app_key`, `app_secret`, `username`, `password` from bKash developer portal
3. Use bKash Checkout URL API — do NOT store card/PIN data yourself
4. Flow: Create Payment → Get paymentURL → Redirect user → Execute Payment on callback
5. Always verify payment status via Query Payment API before fulfilling order
6. bKash charges 1.5% per transaction + monthly fees
7. For small platforms: use SSL Commerz which includes bKash — avoids direct API complexity

Cash on Delivery (COD) — MOST IMPORTANT

COD is 70–80% of all orders in Bangladesh. You MUST get this right. Key risk: fake orders (wrong address, buyer never home). Mitigation strategies:

- Require phone number confirmation via OTP before COD order is accepted
- Limit new buyers to 1-2 COD orders until they have delivery history
- Implement address validation — verify district/upazila combinations
- Add a delivery attempt counter — cancel after 3 failed attempts
- Charge partial advance payment for orders above **■5,000** (reduces fraud)
- Block buyers with >2 failed delivery attempts from COD
- Partner with couriers that offer COD attempt confirmation via SMS

7. Delivery Partner Integration

Integrate at least 2 couriers at launch. Never depend on a single courier — during peak seasons (Eid, 11.11), couriers get overwhelmed. All courier APIs are REST-based with JSON.

Steadfast Courier

Base URL: steadfast-courier.com/api

- Best for ecommerce platforms — has a dedicated merchant API
- Endpoint: POST `/api/v1/create_order`
- Required fields: invoice, recipient_name, recipient_phone, recipient_address, cod_amount
- Returns: consignment_id (tracking ID)
- Track: GET `/api/v1/status_by_consignment/{id}`
- Price: ■110 inside Dhaka, ■150 outside, ■50 extra for COD collection
- Bulk status check: POST `/api/v1/bulk_status_check` (up to 500 IDs)
- Webhook: configure delivery status webhooks in Steadfast merchant portal

Pathao Courier

Base URL: api-hermes.pathao.com

- Largest courier in Bangladesh — very reliable for Dhaka
- Auth: POST `/aladdin/api/v1/issue-token` (get access_token)
- Create order: POST `/aladdin/api/v1/orders`
- Required fields: store_id, recipient_name, recipient_phone, recipient_address, delivery_type
- Delivery types: 48 (Regular), 12 (Express), Sameday
- Track: GET `/aladdin/api/v1/orders/{consignment_id}/info`
- Rate: POST `/aladdin/api/v1/merchant/price-plan` (calculate delivery fee)

Redx

Base URL: openapi.redx.com.bd

- Strong coverage outside Dhaka — good for nationwide reach
- Create parcel: POST `/api/v1.0.0/parcel`
- Required: customer_name, customer_phone, delivery_area, cash_collection_amount
- Get areas: GET `/api/v1.0.0/areas` (for dropdown in seller form)
- Track: GET `/api/v1.0.0/parcel/track/{tracking_id}`
- Good for: fashion, electronics, groceries outside major cities

Paperfly

Base URL: paperfly.com.bd API

- Specialized in ecommerce — good for small parcels and documents
- Create shipment via REST API or bulk CSV upload
- Supports: Dhaka, Chittagong, Sylhet, Rajshahi zones

- COD collection: up to ■20,000 per consignment
- API auth: API key in header X-API-KEY

Delivery Rate Logic

Show estimated delivery fee at checkout. Logic: check product weight, seller's location, buyer's district. Inside Dhaka = lower rate. Outside Dhaka = higher rate. Multiple sellers in one order = combine or split shipments.

8. Authentication & Security

- JWT access token: 15-minute expiry. Refresh token: 30-day expiry (stored in httpOnly cookie)
- Phone-based OTP registration (6-digit, 5-minute expiry, rate-limited to 3 per hour)
- bcrypt password hashing (cost factor 12)
- Rate limiting: 100 req/min per IP (use Redis + express-rate-limit)
- CORS: whitelist your frontend domains only
- Helmet.js: set security HTTP headers (XSS protection, content-type sniff prevention)
- SQL injection prevention: use parameterized queries (TypeORM/Prisma handles this)
- File upload: validate MIME type server-side, max 5MB per image, virus scan optional
- Payment webhook verification: validate SSL Commerz hash/signature on every IPN call
- Admin actions: require re-authentication (password confirm) for destructive actions
- Seller KYC: store NID + trade license scans in private S3 bucket (not public URLs)
- HTTPS everywhere: enforce SSL via Cloudflare or Nginx

Fraud Prevention

- Track buyer delivery success rate — flag accounts below 80%
- Limit COD to ■5,000 for new buyers (first 3 orders)
- Detect multiple accounts from same phone/device fingerprint
- Flash sale: one purchase per user per product variant
- Review gate: only verified buyers who received the product can review
- Seller payout hold: 7-day hold after delivery before releasing payment
- Monitor for duplicate reviews from same IP or device

9. Search & Product Discovery

Use Meilisearch for instant, typo-tolerant search. Index products on creation/update. Sync via background job.

- Install: `npm install meilisearch`
- Index name: 'products'
- Searchable attributes: `name_en`, `name_bn`, `category`, `brand`, `tags`
- Filterable attributes: `category_id`, `seller_id`, `price`, `rating`, `in_stock`, `is_featured`
- Sortable attributes: `price`, `created_at`, `sales_count`, `rating`
- Ranking rules: `words`, `typo`, `proximity`, `attribute`, `sort`, `exactness`
- Sync: on product create/update, push to Meilisearch via NestJS event (EventEmitter or Bull queue)
- Search endpoint: `GET /products/search?q=&category;=&min;_price=&max;_price=&sort;=`
- Autocomplete: separate lightweight index with product names only
- Bangla: Meilisearch supports Unicode — Bangla search works natively

Homepage Algorithm

- Featured products: manually curated by admin (`is_featured = true`)
- Flash sale: products in active `flash_sales` with discounted price + countdown
- Trending: ordered by `views_count DESC` + `sales_count DESC` (last 7 days)
- Recommended (Phase 2): collaborative filtering based on purchase history
- New arrivals: ordered by `created_at DESC`, limit 20
- Category banners: admin-uploaded promotional banners per category

13. AI Coding Prompts (Copy-Paste Ready)

Use these prompts with Claude, ChatGPT, or GitHub Copilot to generate code for each module. Always add your specific project context when using them.

NestJS Auth Module

You are building a NestJS authentication module for a Bangladesh ecommerce platform. Create: (1) AuthModule with JWT strategy, (2) AuthController with endpoints: POST /auth/register, POST /auth/login, POST /auth/otp/send, POST /auth/otp/verify, POST /auth/refresh, GET /auth/me. (3) Use bcrypt for password hashing, (4) JWT access token 15min expiry, refresh token 30 days, (5) OTP: 6-digit code, store in Redis with 5-min TTL, rate-limit 3 OTPs per phone per hour. (6) AuthGuard decorator for protected routes. Use TypeORM with PostgreSQL. User entity: id (UUID), name, phone (unique), email, password_hash, role (buyer/seller/admin).

Product Catalog API

Build a NestJS Products module for a multi-vendor ecommerce platform. Include: (1) Product entity with all fields from schema (id, seller_id, category_id, name_en, name_bn, slug, description, base_price, sale_price, stock_quantity, status, is_featured). (2) ProductsController: GET /products (with filters: category, min_price, max_price, in_stock, seller_id, sort: price_asc/price_desc/newest/popular), GET /products/:slug, POST /products (seller only), PATCH /products/:id (seller only), DELETE /products/:id. (3) Auto-generate slug from name_en using slugify. (4) On create/update, sync to Meilisearch index. (5) Image upload to S3 – return image URLs. Max 8 images per product.

Checkout & Order Flow

Create a NestJS OrdersModule for a Bangladesh ecommerce checkout. Flow: (1) POST /orders/checkout – takes: items (product_id, variant_id, qty), delivery_address_id, payment_method (cod/bkash/nagad/card), coupon_code. (2) Validate stock for each item. Lock stock immediately. (3) Calculate: subtotal, delivery_fee (by courier zone), discount (coupon), total. (4) Create Order + OrderItems records in a database transaction. (5) If payment_method is COD: set payment_status = unpaid, order status = confirmed. (6) If payment_method is bkash/card: initiate SSL Commerz payment and return redirect URL. (7) On payment success callback: update payment_status = paid, trigger order confirmation notification. (8) Send SMS notification to buyer and seller via SSL Wireless API.

SSL Commerz Payment Service

Create a NestJS PaymentsService integrating SSL Commerz for a Bangladesh ecommerce site. Include: (1) initiateSslCommerzPayment(order) method that calls SSL Commerz init API and returns GatewayPageURL. (2) verifySslCommerzPayment(val_id) that calls validation endpoint. (3) handleSuccess(req) – called on success redirect, verify payment, update order. (4) handleIPN(req) – called on IPN webhook, re-verify and update if not already updated. (5) Store gateway_transaction_id and full gateway_response JSONB in payments table. Env vars: SSLCOMMERZ_STORE_ID, SSLCOMMERZ_STORE_PASSWD, SSLCOMMERZ_IS_LIVE (true/false). Use sslcommerz-lts npm package. Handle fail and cancel callbacks too.

Steadfast Courier Integration

Build a NestJS DeliveryService that integrates with Steadfast Courier API. Include: (1) createShipment(order) – POST to https://portal.steadfast.com.bd/api/v1/create_order with: invoice (order_number), recipient_name, recipient_phone, recipient_address, cod_amount (0 if prepaid, full amount if COD), note, weight. Returns consignment_id. (2) trackShipment(consignment_id) – GET status from Steadfast. (3) bulkTrackShipments(ids[]) – bulk status check. (4) calculateDeliveryFee(district) – return 110 if district is in Dhaka zones, 150 otherwise. Auth: API key in header 'Api-Key'. Store STEADFAST_API_KEY in .env. Save consignment_id to shipments table. Webhook: handle status updates from Steadfast.

Meilisearch Product Search

Create a NestJS SearchService using Meilisearch for product search. Include: (1) syncProduct(product) – index or update a product document in Meilisearch 'products' index. (2) deleteProduct(id) – remove from index. (3) search(query, filters) – search with: text query, category filter, price range filter, in_stock filter, sorting options (price_asc, price_desc, newest, popular). Return paginated results. (4) autocomplete(query) – return 5 product name suggestions. (5) Configure index on startup: searchable attrs [name_en, name_bn, tags], filterable attrs [category_id, base_price, in_stock], sortable attrs [base_price, created_at]. Use MeiliSearch npm client. MEILISEARCH_URL and MEILISEARCH_MASTER_KEY in .env.

Next.js Product Page

Create a Next.js 14 App Router product detail page at /product/[slug]. Use server-side rendering (async page component). Include: (1) Fetch product by slug from API, return 404 if not found. (2) ProductGallery component – main image + thumbnail row, click thumbnail to change main. (3) VariantSelector – show color/size options as clickable chips, update price on selection. (4) AddToCartButton – updates cart context, shows success toast. (5) Price display: show sale_price if set (with strikethrough on base_price), calculate discount percentage badge. (6) Stock indicator: In Stock (green) / Only N left (amber) / Out of Stock (red). (7) ReviewSection – show star distribution, list reviews with pagination. (8) RelatedProducts – 4 products from same category. (9) Generate metadata (title, description, og:image) for SEO. Use Tailwind CSS. Support Bangla language toggle.

Seller Dashboard

Build a Next.js 14 seller dashboard at /seller/dashboard. Protected by seller auth. Include these sections: (1) Stats overview cards: Today's orders, Total revenue, Pending orders, Products listed, Average rating. (2) Recent orders table: order number, buyer name, items, total, status, action buttons. (3) Revenue chart: line chart showing daily revenue for last 30 days. (4) Products table: name, stock, price, status (active/paused), edit/delete actions. (5) Quick actions: Add Product, View Payouts, Download Invoice. (6) Notification badge for new orders. Use React Query for data fetching with 30-second refetch interval. Use Recharts for the revenue chart. Tailwind CSS for styling.

14. Infrastructure & DevOps

Recommended Server Setup (Launch Phase)

Server	Spec	Purpose	Cost/mo
App Server	4 vCPU, 8GB RAM	NestJS API + Next.js	~\$48 (DO)
DB Server	2 vCPU, 4GB RAM	PostgreSQL + Redis	~\$24 (DO)
Search Server	2 vCPU, 4GB RAM	Meilisearch	~\$24 (DO)
Cloudflare	Free tier	CDN + SSL + DDoS	\$0
S3 / Wasabi	Pay per use	File storage	~\$5–20
Total			~\$100–120/mo to start

Docker Compose (Development)

```

version: '3.9'

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: ecommerce_bd
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: yourpassword
    ports: ['5432:5432']
    volumes: [postgres_data:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

  meilisearch:
    image: getmeili/meilisearch:v1.4
    ports: ['7700:7700']
    environment:
      MEILI_MASTER_KEY: your_master_key_here

  api:
    build: ./apps/api

```

```
ports: ['3001:3001']
depends_on: [postgres, redis, meilisearch]
env_file: .env

web:
build: ./apps/web
ports: ['3000:3000']
env_file: .env

volumes:
postgres_data:
```


15. Environment Variables & Config

```
# Database
DATABASE_URL=postgresql://admin:password@localhost:5432/ecommerce_bd
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=your_very_long_random_secret_here_min_32_chars
JWT_EXPIRES_IN=15m
JWT_REFRESH_SECRET=another_very_long_random_refresh_secret
JWT_REFRESH_EXPIRES_IN=30d

# Search
MEILISEARCH_URL=http://localhost:7700
MEILISEARCH_MASTER_KEY=your_meilisearch_master_key

# Storage
AWS_ACCESS_KEY_ID=your_aws_key
AWS_SECRET_ACCESS_KEY=your_aws_secret
AWS_REGION=ap-southeast-1
AWS_BUCKET_NAME=your-ecommerce-bucket
CLOUDFRONT_DOMAIN=https://cdn.yoursite.com.bd

# Payments
SSLCOMMERZ_STORE_ID=your_store_id
SSLCOMMERZ_STORE_PASSWD=your_store_password
SSLCOMMERZ_IS_LIVE=false # change to true in production
BKASH_APP_KEY=your_bkash_app_key
BKASH_APP_SECRET=your_bkash_app_secret
BKASH_USERNAME=your_bkash_username
BKASH_PASSWORD=your_bkash_password

# Couriers
STEADFAST_API_KEY=your_steadfast_api_key
STEADFAST_SECRET_KEY=your_steadfast_secret
PATHAO_CLIENT_ID=your_pathao_client_id
PATHAO_CLIENT_SECRET=your_pathao_client_secret
REDX_API_TOKEN=your_redx_token

# SMS
```

```
SSL_WIRELESS_SID=your_ssl_wireless_sid
```

SSL_WIRELESS_SENDERID=YourBrand

[illegible]

```
SENDGRID_API_KEY=SG.your_sendgrid_key
```

FROM_EMAIL=noreply@yoursite.com.bd

[illegible]

```
FIREBASE_PROJECT_ID=your_project_id
```

```
FIREBASE_PRIVATE_KEY=your_private_key
```

```
FIREBASE_CLIENT_EMAIL=firebase@your-project.iam.gserviceaccount.com
```

App

NODE_ENV=development

PORT=3001

```
FRONTEND_URL=http://localhost:3000
```

```
API_URL=http://localhost:3001
```

17. Launch Checklist

Backend

- ☐ All API endpoints tested with Postman/Insomnia collection
- ☐ Database migrations run cleanly on fresh install
- ☐ All environment variables documented and set in production
- ☐ SSL Commerz in LIVE mode (not sandbox)
- ☐ bKash / Nagad merchant accounts activated
- ☐ Steadfast + Pathao merchant accounts active
- ☐ SMS gateway (SSL Wireless) active and sender ID approved
- ☐ Redis configured with password in production
- ☐ Database backups automated (daily at minimum)
- ☐ API rate limiting active
- ☐ CORS configured for production domain only

Frontend

- ☐ All pages tested on mobile (Chrome DevTools + real device)
- ☐ Bangla language toggle working correctly
- ☐ Images served via CDN (Cloudflare)
- ☐ All images in WebP format with lazy loading
- ☐ Lighthouse score > 80 on mobile
- ☐ SEO: meta tags, og:image, sitemap.xml, robots.txt
- ☐ Google Analytics / Meta Pixel installed
- ☐ 404 and 500 error pages customised
- ☐ Cookie consent banner (GDPR / BD data protection)

Business

- ☐ Company registered (sole proprietorship or Ltd.)
- ☐ Trade license obtained from local city corporation
- ☐ Bank account opened (business account)
- ☐ Payment gateway merchant agreement signed
- ☐ Refund & return policy page live
- ☐ Terms of Service and Privacy Policy pages live
- ☐ Customer support channel ready (Facebook Page / WhatsApp / Live Chat)
- ☐ At least 20 verified sellers onboarded before launch
- ☐ At least 500 products listed before launch
- ☐ Courier agreements signed with at least 2 couriers

Security

- ☐ HTTPS enforced on all domains
- ☐ SQL injection tested (use Prisma/TypeORM — parameterized)
- ☐ XSS prevention: sanitize all user inputs
- ☐ File upload: MIME type validation, size limits
- ☐ Admin panel on separate subdomain (admin.yoursite.com.bd)
- ☐ Seller KYC documents in private S3 bucket (not public)
- ☐ Payment webhook signatures verified
- ☐ Penetration test or security audit done

Good luck building! ■

This document covers the full technical specification. Start with Phase 1 only — auth, products, checkout, COD payment, and one courier integration. Get your first 100 orders before building Phase 2. Validate demand before scaling infrastructure.

Bangladesh Ecommerce Platform — Technical Specification v1.0 | Generated by AI | Verify all API endpoints with official courier/payment documentation before production use.